

Geometric Control with L1 Adaptive Augmentation for Acrobatic Quadrotor Maneuvers

Sourav Selvaraj

Department of Robotics Engineering
Worcester Polytechnic Institute, MA, USA
Email: sselvaraj@wpi.edu

Abstract—This work presents the design and implementation of a geometric attitude and position controller with L1 adaptive augmentation for aggressive quadrotor flight. The controller is implemented on a Crazyflie drone in a ROS 2 + Ignition Gazebo simulation environment. The proposed architecture operates on the configuration space $SE(3) = SO(3) \times \mathbb{R}^3$ and avoids Euler-angle singularities by formulating the attitude dynamics directly on $SO(3)$, following the geometric control framework of Lee *et al.*. A smooth min-jerk trajectory brings the vehicle from an arbitrary initial state to a hover at (2, 2, 2) m, after which a time-parameterized, zero-radius 2π roll flip is executed in 0.2 s, followed by a recovery and hover-hold phase. The nominal geometric controller is augmented by an L1 adaptive layer acting on all three body-rate channels (p, q, r) to mitigate modeling errors and injected step torque disturbances in tracking mode. The full controller is implemented as a single ROS 2 node in C++, including mode logic, torque allocation, and detailed logging for post-processing. Simulation results demonstrate stable tracking, successful completion of the flip maneuver, and significantly improved disturbance rejection with L1 augmentation compared to the baseline geometric controller.

I. INTRODUCTION

Classical cascaded PID controllers or linearized models around hover are effective for small-angle flight, but they suffer from performance degradation or even failure when the vehicle performs large attitude excursions or full 360° rotations. Representing attitude with Euler angles introduces singularities (gimbal lock) and complicated coupling, which makes aggressive maneuvers difficult to control robustly.

To address these limitations, geometric control techniques formulate the attitude dynamics directly on the Special Orthogonal Group $SO(3)$ and the full configuration on $SE(3)$, providing globally valid attitude representations and Lyapunov-based stability guarantees for coupled position and attitude dynamics [1]. Geometric controllers have been experimentally shown to enable backflips and other acrobatic maneuvers on miniature quadrotors [2].

However, purely geometric controllers can still be sensitive to unmodeled dynamics, parameter errors, and external disturbances. L1 adaptive control has emerged as a powerful framework that provides fast adaptation with guaranteed robustness margins by filtering the adaptive signal through a low-pass element [3], [4]. When applied to quadrotors, L1 augmentation can compensate for uncertainties and improve robustness without sacrificing transient performance.

This report presents a unified controller that combines geometric tracking on $SE(3)$ with an L1 adaptive layer acting on the body-rate channels. The controller is implemented in ROS 2 as a single C++ node that:

- tracks a min-jerk position and yaw trajectory to (2, 2, 2) m,
- triggers a zero-radius roll flip (0 to 2π) about the body x -axis over 0.2 s,
- performs a recovery phase to upright hover, and
- transitions to a hover-hold mode.

A constant matched step torque disturbance is injected during the trajectory tracking phase to evaluate disturbance rejection with and without L1 adaptation. The report focuses not only on the control design, but also on clear naming and implementation details so that the C++ code and the equations can be read side-by-side.

II. QUADROTOR MODELING ON $SE(3)$

This section summarizes the continuous-time dynamics of the quadrotor in a form that matches the implementation.

A. State and Configuration

The quadrotor is modeled as a rigid body with configuration $R \in SO(3)$ and position $p \in \mathbb{R}^3$ in the world frame:

$$x = (p, v, R, \Omega), \quad (1)$$

where

- $p = [x, y, z]^\top$ is the center-of-mass position in the inertial frame,
- $v = \dot{p}$ is the linear velocity in the inertial frame,
- $R \in SO(3)$ maps body-frame vectors to the inertial frame,
- $\Omega = [\Omega_x, \Omega_y, \Omega_z]^\top$ is the body-frame angular velocity.

B. Dynamics

The dynamics on $SE(3)$ are given by [1]

$$\dot{p} = v, \quad (2)$$

$$m\dot{v} = mge_3 - fRe_3, \quad (3)$$

$$\dot{R} = R\hat{\Omega}, \quad (4)$$

$$J\dot{\Omega} + \Omega \times (J\Omega) = \tau, \quad (5)$$

where:

- m is the total mass,

- $J \in \mathbb{R}^{3 \times 3}$ is the inertia matrix about the body origin,
- g is gravitational acceleration,
- $e_3 = [0, 0, 1]^\top$,
- $f \in \mathbb{R}$ is the total thrust along the body $-z$ axis,
- $\tau = [\tau_x, \tau_y, \tau_z]^\top$ is the body torque vector,
- $\hat{\Omega}$ is the skew-symmetric matrix such that $\hat{\Omega}x = \Omega \times x$.

For the Crazyflie-like platform considered here, the mass and inertia are

$$m = 0.025 + 4 \cdot 0.0008 \approx 0.0282 \text{ kg}, \quad (6)$$

$$J = \text{diag}(1.65, 1.67, 2.93) \times 10^{-5} \text{ kg m}^2. \quad (7)$$

The arm length is $d = \sqrt{0.031^2 + 0.031^2} \approx 0.0438 \text{ m}$, corresponding to the distance from the body origin to each rotor.

C. Rotor Forces and Allocation

Each rotor $i \in \{1, 2, 3, 4\}$ produces thrust

$$f_i = k_f \omega_i^2, \quad (8)$$

and reaction torque proportional to $k_m \omega_i^2$ with appropriate sign. The total thrust and torques can be written as

$$U = \begin{bmatrix} f \\ \tau_x \\ \tau_y \\ \tau_z \end{bmatrix} = A \begin{bmatrix} f_1 \\ f_2 \\ f_3 \\ f_4 \end{bmatrix}, \quad (9)$$

where the allocation matrix A for the rotor ordering (front-left, front-right, rear-right, rear-left) is

$$A = \begin{bmatrix} 1 & 1 & 1 & 1 \\ d & -d & -d & d \\ -d & -d & d & d \\ -k_m & k_m & -k_m & k_m \end{bmatrix}. \quad (10)$$

The controller computes the desired U , then recovers the rotor forces via

$$[f_1, f_2, f_3, f_4]^\top = A^{-1}U, \quad (11)$$

and back-solves for $\omega_i = \sqrt{f_i/k_f}$ subject to saturation constraints. In the implementation, each rotor is allowed to produce up to six times the hover thrust, subject to the maximum rotor speed $3\omega_{\text{hover}}$.

III. NOTATION AND IMPLEMENTATION VARIABLE NAMING

To make the C++ implementation easy to read next to the equations, this section summarizes the naming conventions used in the ROS 2 node. The code uses a trailing underscore for member variables (for example, `m_`), and closely follows the mathematical notation.

A. State and Reference Variables

Table I shows the main state and reference variables.

The goal position $(2, 2, 2)$ and yaw are stored as:

- $p_g \leftrightarrow \text{x_goal_}$,
- $\psi_g \leftrightarrow \text{yaw_goal_}$.

The initial state of the min-jerk trajectory is kept in `x_start_` and `yaw_start_`.

TABLE I: State and reference variables used in the controller.

Quantity	Symbol
Position (world)	$p = [x, y, z]^\top$
Velocity (world)	v
Attitude	$R \in SO(3)$
Body rates	$\Omega = [p, q, r]^\top$
Desired position	p_d
Desired velocity	v_d
Desired acceleration	a_d
Desired yaw	ψ_d
Desired yaw rate	$\dot{\psi}_d$
Desired yaw acceleration	$\ddot{\psi}_d$

TABLE II: Physical parameters and controller gains used in the implementation.

Quantity	Symbol	Value
Mass	m	0.0282 kg
Arm length	d	0.0438 m
Thrust coeff.	k_f	$1.28 \times 10^{-8} \text{ N s}^2/\text{rad}^2$
Moment coeff.	k_m	$1.6 \times 10^{-2} \text{ N m s}^2/\text{rad}^2$
Gravity	g	9.81 m/s ²
Inertia matrix	J	$\text{diag}(1.65, 1.67, 2.93) \times 10^{-5} \text{ kg m}^2$
Position gains	K_p	$\text{diag}(1.60, 1.80, 0.4512)$
Velocity gains	K_v	$\text{diag}(0.12, 0.12, 0.2256)$
Attitude gains	K_R	$\text{diag}(0.026, 0.010, 0.0055)$
Rate gains	K_Ω	$\text{diag}(0.0050, 0.0050, 0.000586)$

B. Physical Parameters and Gains

Table II summarizes the main physical parameters and gain vectors.

All of these are initialized once in the constructor and then treated as constants inside the control loop.

C. Mode and L1 Variables

The controller mode is represented by an enum class `Mode` with values `TRACK`, `FLIP`, `RECOVER`, and `HOLD`, stored in `mode_`. A compact integer flag (0–3) is logged to the CSV file for offline analysis.

For the L1 adaptive layer:

- $\hat{x}$ (predictor state for $[p, q, r]$) $\leftrightarrow$ `l1_xhat_pqr_`,
- $\hat{\sigma}$ (raw disturbance estimate) $\leftrightarrow$ `l1_sigma_hat_`,
- σ_f (filtered estimate) $\leftrightarrow$ `l1_sigma_filt_`,
- u_{ad} (adaptive torque) $\leftrightarrow$ `l1_u_ad_log_`,
- predictor pole $a \leftrightarrow$ `l1_a_`,
- adaptation gains $\Gamma \leftrightarrow$ `l1_gamma_` (vector),
- filter cutoff $\omega_c \leftrightarrow$ `l1_omega_c_`,
- bounds $\sigma_{\text{max}} \leftrightarrow$ `l1_sigma_max_vec_`,
- inertia diagonal $J_{\text{diag}} \leftrightarrow$ `Jdiag_`.

This one-to-one mapping means that whenever a symbol appears in the equations (for example, σ_{max}), the corresponding C++ variable can be found immediately and vice versa.

IV. GEOMETRIC TRACKING CONTROL ON $SE(3)$

This section describes how the controller computes thrust and torque from the state and trajectory references.

A. Tracking Errors

Let $(p_d, v_d, R_d, \Omega_d)$ denote the desired state. The tracking errors are

$$e_p = p - p_d, \quad (12)$$

$$e_v = v - v_d. \quad (13)$$

The attitude error $e_R \in \mathbb{R}^3$ is defined as [1]

$$e_R = \frac{1}{2}(R_d^\top R - R^\top R_d)^\vee, \quad (14)$$

where $(\cdot)^\vee$ maps a skew-symmetric matrix to $\mathbb{R}^3$. The angular velocity error is

$$e_\Omega = \Omega - R^\top R_d \Omega_d. \quad (15)$$

In the implementation, the attitude error is computed from the relative rotation

$$R_{\text{rel}} = R_d^\top R, \quad (16)$$

converted to a quaternion (q_w, q_x, q_y, q_z) , and mapped to

$$e_R = 2 \text{sign}(q_w) [q_x, q_y, q_z]^\top, \quad (17)$$

which remains non-zero at 180° rotations and is numerically robust.

B. Nominal Control Law

The nominal geometric controller for thrust and torque follows the structure of [1]. The desired thrust direction is derived from a virtual force

$$A = -K_p e_p - K_v e_v + m g e_3 + m a_d, \quad (18)$$

where $K_p, K_v \in \mathbb{R}^{3 \times 3}$ are diagonal gains and a_d is the desired acceleration. The desired body z -axis is

$$b_{3d} = \begin{cases} \frac{A}{\|A\|}, & \|A\| > \epsilon, \\ e_3, & \text{otherwise.} \end{cases} \quad (19)$$

The desired yaw ψ_d defines a horizontal reference direction

$$b_{1d} = [\cos \psi_d, \sin \psi_d, 0]^\top. \quad (20)$$

Then

$$b_{2d} = \frac{b_{3d} \times b_{1d}}{\|b_{3d} \times b_{1d}\|}, \quad (21)$$

$$b_{1c} = b_{2d} \times b_{3d}, \quad (22)$$

and the desired attitude is

$$R_d = [b_{1c} \ b_{2d} \ b_{3d}]. \quad (23)$$

The desired thrust magnitude is

$$f = A^\top R e_3, \quad (24)$$

clamped between $u_{\min}$ and $u_{\max}$ and additionally constrained by rotor limits.

The body torque is

$$\tau_{\text{geo}} = \begin{bmatrix} -k_{R,x} e_{R,x} - k_{\Omega,x} e_{\Omega,x} \\ -k_{R,y} e_{R,y} - k_{\Omega,y} e_{\Omega,y} \\ k_{R,z} e_{R,z} + k_{\Omega,z} e_{\Omega,z} \end{bmatrix} + \tau_{\text{ff}} + \tau_{\text{gyro}}, \quad (25)$$

where the gains are axis-specific and

$$\tau_{\text{gyro}} = \Omega \times (J\Omega), \quad (26)$$

$$\tau_{\text{ff}} = J \left(R^\top R_d \dot{\Omega}_d - \Omega \times (R^\top R_d \Omega_d) \right), \quad (27)$$

providing feedforward compensation for desired angular acceleration and the Coriolis term.

C. Mode-Dependent Thrust and Gains

The controller operates in four modes:

- 1) **TRACK**: Full $SE(3)$ geometric tracking to $p_d(t)$ and $R_d(t)$.
- 2) **FLIP**: Zero-radius 2π roll flip about the body x axis.
- 3) **RECOVER**: Altitude PD plus upright attitude regulation.
- 4) **HOLD**: $SE(3)$ tracking to a fixed hover at $p = (2, 2, 2)$.

In TRACK and HOLD, thrust is generated from the $SE(3)$ law above. In RECOVER, a scalar z -axis PD controller computes a vertical force

$$u_z = m g - k_z e_z - k_{vz} e_{vz}, \quad (28)$$

which is tilt-compensated using the projection of $R e_3$ on e_3 . Moment limits are mode-dependent:

- stronger roll/pitch limits in FLIP,
- reduced but still agile limits in TRACK/RECOVER,
- common yaw limit across modes.

V. TRAJECTORY GENERATION AND MODE LOGIC

This section explains how the desired trajectories are built and how the controller switches between modes.

A. Min-Jerk Position and Yaw Trajectory

When the controller starts, a min-jerk trajectory moves the quadrotor from its initial state (p_0, ψ_0) to $(p_g, \psi_g) = (2, 2, 2, 0^\circ)$ in fixed time $T = 7$ s. The scalar time-scaling $s = t/T$ is used to construct

$$q(s) = 10s^3 - 15s^4 + 6s^5, \quad (29)$$

$$\dot{q}(s) = 30s^2 - 60s^3 + 30s^4, \quad (30)$$

$$\ddot{q}(s) = 60s - 180s^2 + 120s^3. \quad (31)$$

The desired position, velocity, and acceleration are

$$p_d(t) = p_0 + q(s)(p_g - p_0), \quad (32)$$

$$v_d(t) = \dot{q}(s)(p_g - p_0), \quad (33)$$

$$a_d(t) = \ddot{q}(s)(p_g - p_0), \quad (34)$$

and yaw follows the same profile from ψ_0 to ψ_g .

B. Flip Trajectory on $SO(3)$

Once the min-jerk trajectory is complete and the tracking errors satisfy

$$\|e_p\|_\infty < 0.02 \text{ m}, \quad \|e_R\| < 2^\circ, \quad (35)$$

the controller switches to FLIP mode. A smooth roll profile $\theta(t)$ from 0 to 2π over $T_{\text{flip}} = 0.2$ s is defined using a cosine-based smooth step:

$$s = \text{clip}\left(\frac{t - t_{\text{flip,start}}}{T_{\text{flip}}}, 0, 1\right), \quad (36)$$

$$\theta(s) = \frac{1}{2}\Theta_{\text{tot}}(1 - \cos(\pi s)), \quad (37)$$

where $\Theta_{\text{tot}} = 2\pi$. The corresponding rates are

$$\dot{\theta}(s) = \frac{1}{2}\Theta_{\text{tot}}\frac{\pi}{T_{\text{flip}}}\sin(\pi s), \quad (38)$$

$$\ddot{\theta}(s) = \frac{1}{2}\Theta_{\text{tot}}\frac{\pi^2}{T_{\text{flip}}^2}\cos(\pi s). \quad (39)$$

A base attitude R_{base} is generated from the current $SE(3)$ virtual force A and a fixed yaw reference, as in TRACK mode. The desired flip attitude is

$$R_d(t) = R_{\text{base}}(t) \exp(\hat{e}_1 \theta(t)), \quad (40)$$

where $e_1 = [1, 0, 0]^\top$ and $\exp(\cdot)$ is the matrix exponential. The desired body rates and accelerations are approximated by

$$\Omega_d(t) = [\dot{\theta}(t), 0, 0]^\top, \quad (41)$$

$$\dot{\Omega}_d(t) = [\ddot{\theta}(t), 0, 0]^\top. \quad (42)$$

Throughout the flip, the center-of-mass reference is held at $p_g = (2, 2, 2)$ and $v_d = a_d = 0$, i.e., a zero-radius flip.

C. Recover and Hold

After T_{flip} seconds, the controller switches to RECOVER mode. The altitude is regulated using a dedicated z -PD loop, and an upright attitude with frozen yaw reference is commanded. The RECOVER mode transitions to HOLD when:

- roll and pitch are within $\pm 5^\circ$,
- body rates are small ($\|\Omega\|_\infty < 0.5$ rad/s),
- altitude error is below 5 cm,
- a minimum RECOVER time has elapsed.

In HOLD, a standard $SE(3)$ law tracks a fixed hover at p_g with constant yaw.

VI. L1 ADAPTIVE AUGMENTATION OF BODY-RATE DYNAMICS

A. Matched Uncertainty Model

The rotational dynamics with matched disturbances can be written as

$$J\dot{\Omega} = \tau_{\text{cmd}} + \sigma(t) - \Omega \times (J\Omega), \quad (43)$$

where $\sigma(t)$ collects unknown torques due to modeling errors, aerodynamic effects, or injected disturbances. The geometric controller provides a nominal torque τ_{geo} , and the L1 layer adds adaptive compensation u_{ad} :

$$\tau_{\text{cmd}} = \tau_{\text{geo}} + u_{\text{ad}} + \tau_{\text{dist}}, \quad (44)$$

where τ_{dist} is an explicit test disturbance (see Sec. VII).

B. Predictor Dynamics

For each axis $i \in \{x, y, z\}$ with inertia J_i , the predictor state $\hat{x}_i$ approximates the body rate Ω_i :

$$\dot{\hat{x}}_i = -a\hat{x}_i + \frac{1}{J_i}(\tau_{\text{geo},i} + u_{\text{ad},i} + \hat{\sigma}_i), \quad (45)$$

where $a > 0$ is a design pole and $\hat{\sigma}_i$ is the disturbance estimate. In vector form,

$$\dot{\hat{\mathbf{x}}} = -a\hat{\mathbf{x}} + J_{\text{diag}}^{-1}(\tau_{\text{geo}} + \mathbf{u}_{\text{ad}} + \hat{\boldsymbol{\sigma}}), \quad (46)$$

with $J_{\text{diag}} = \text{diag}(J_x, J_y, J_z)$.

In the implementation, a common pole $a = 10$ is used for all axes.

C. Adaptation and Projection

The prediction error $e_p = \Omega - \hat{\mathbf{x}}$ drives the adaptation law

$$\dot{\hat{\boldsymbol{\sigma}}} = \boldsymbol{\Gamma} \text{sat}_\Delta(e_p), \quad (47)$$

where $\boldsymbol{\Gamma}$ is a diagonal adaptation gain matrix and $\text{sat}_\Delta(\cdot)$ includes a small deadzone to reduce noise sensitivity. Specifically,

$$\boldsymbol{\Gamma} = \text{diag}(110, 260, 150), \quad (48)$$

and a deadzone of $0.2^\circ/\text{s}$ (converted to rad/s) is applied per axis.

Each component $\hat{\sigma}_i$ is projected onto a bounded interval

$$|\hat{\sigma}_i| \leq \sigma_{\text{max},i}, \quad (49)$$

with

$$\boldsymbol{\sigma}_{\text{max}} = [0.0030, 0.0030, 0.0015]^\top \text{ N m}. \quad (50)$$

This ensures that the adaptive torque respects known disturbance bounds. Unlike earlier variants, the current implementation enables adaptation on all three axes (p, q, r), with a slightly tighter bound on yaw.

D. Low-Pass Filtering and Adaptive Torque

To obtain the actual compensation used by the controller, $\hat{\boldsymbol{\sigma}}$ is passed through a first-order low-pass filter,

$$\dot{\boldsymbol{\sigma}}_f = -\omega_c \boldsymbol{\sigma}_f + \omega_c \hat{\boldsymbol{\sigma}}, \quad (51)$$

where $\omega_c = 10$ rad/s is the cutoff frequency. The adaptive torque is finally

$$\mathbf{u}_{\text{ad}} = -\boldsymbol{\sigma}_f. \quad (52)$$

The combination of fast adaptation and low-pass filtering matches the L1 architecture: adaptation is fast, but the control channel is bandwidth-limited to preserve robustness.

To avoid corrupting transient behavior during large errors (for example, immediately before or after the flip), the adaptation is *frozen* when both $\|e_R\|$ and $\|e_\Omega\|$ fall below small thresholds (1.5° and $5^\circ/\text{s}$, respectively). In that case, the predictor and estimates are not updated, but the last filtered $\boldsymbol{\sigma}_f$ is still used.

VII. DISTURBANCE INJECTION AND LOGGING

A constant step disturbance torque is injected in TRACK mode to evaluate robustness. In the current implementation, the disturbance vector is

$$\tau_{\text{dist}} = \begin{bmatrix} 1.0 \times 10^{-3} \\ -1.5 \times 10^{-3} \\ 8.0 \times 10^{-4} \end{bmatrix} \text{ N m}, \quad (53)$$

applied to roll, pitch, and yaw channels, respectively. The disturbance is *zero* outside a short time window and active only for

$$t \in [2.0, 2.8] \text{ s} \quad (54)$$

of the trajectory time in TRACK mode. This makes it a clean step disturbance test: the controller must reject a sudden onset of torque and then handle its removal.

The controller logs data to a CSV file, including:

- time,
- p and p_d , v and v_d ,
- roll, pitch, yaw (actual and desired),
- body rates Ω and Ω_d ,
- thrust f ,
- commanded torques τ_{cmd} ,
- injected disturbance τ_{dist} ,
- adaptive torques u_{ad} ,
- rotor angular velocities ω_i ,
- mode flag (TRACK/FLIP/RECOVER/HOLD),
- disturbance on/off flag.

These logs are used offline to compute error statistics, visualize trajectories, and quantify how much of the injected disturbance is rejected by the L1 layer.

VIII. ROS 2 AND SIMULATION IMPLEMENTATION

This section describes how the controller is integrated into the ROS 2 and Ignition Gazebo stack, and how data flows through the system at runtime.

A. Node Structure and Interfaces

The full controller is implemented as a single ROS 2 node, `GeometricController`, written in C++. The node exposes the following interfaces:

- **Subscriptions**
 - `/crazyflie/odom` (`nav_msgs/Odometry`): provides position, linear velocity, and orientation in the world frame.
 - `/crazyflie/imu` (`sensor_msgs/Imu`): provides body-frame angular velocities used as Ω .
- **Publications**
 - `/crazyflie/motor_cmd` (`actuator_msgs/Actuators`): motor angular velocity commands for the four rotors.
 - `/crazyflie/odom_desired` (`nav_msgs/Odometry`): the desired pose and twist, used only for visualization and plotting.
- **Timer**

- a wall timer running at 200 Hz (period 5 ms), which calls the main control loop.

The controller only starts once both odometry and IMU messages have been received, to avoid using uninitialized state. At that moment, the node records the current state as the start of the min-jerk trajectory and logs the wall-clock time as the controller start time.

B. Control Loop Flow

At every timer tick, the node executes the following sequence:

- 1) **Update reference:** The min-jerk trajectory generator is called to update $p_d(t)$, $v_d(t)$, $a_d(t)$ and $\psi_d(t)$ (and their derivatives) based on the current time and the chosen phase (TRACK, FLIP, RECOVER, or HOLD).
- 2) **Mode logic:** Using the current state, reference, and mode, the node checks the transition conditions:
 - TRACK $\rightarrow$ FLIP when the min-jerk trajectory is complete and the position/attitude errors are small.
 - FLIP $\rightarrow$ RECOVER after the fixed flip duration $T_{\text{flip}} = 0.2$ s.
 - RECOVER $\rightarrow$ HOLD when roll, pitch, body rates, and altitude have converged within specified thresholds.
- 3) **Compute $SE(3)$ control:** Given (p, v, R, Ω) and $(p_d, v_d, a_d, R_d, \Omega_d)$, the node computes the virtual force A , the desired attitude R_d , the thrust f , and the nominal geometric torque τ_{geo} as described in Sections III–V. Mode-dependent thrust and moment limits are applied at this stage.
- 4) **Apply L1 adaptive augmentation:** If L1 is active in the current mode (TRACK, RECOVER, or HOLD; optionally FLIP), the predictor, adaptation law, projection, and low-pass filter are updated based on the body rates and commanded torques. The resulting adaptive torque u_{ad} is added to the nominal torque. When the errors (e_R, e_Ω) are very small, the predictor is frozen and the last filtered estimate is reused.
- 5) **Disturbance injection (step test):** In TRACK mode, if the current trajectory time lies in the window $[2.0, 2.8]$ s, the constant step disturbance τ_{dist} is added for robustness evaluation. Outside this window, no disturbance is injected.
- 6) **Allocation and motor commands:** The final thrust f and torque vector τ_{cmd} are stacked into U , mapped through A^{-1} to rotor thrusts f_i , clamped, and then converted to rotor angular velocities ω_i . The node publishes these on `/crazyflie/motor_cmd`.
- 7) **Logging and visualization:** The node logs the current sample to a CSV file and publishes the desired state on `/crazyflie/odom_desired` so that actual vs. desired trajectories can be visualized in RViz or post-processed with Python.

TABLE III: Tracking error metrics: RMSE and MAE over the full maneuver.

Signal	Unit	RMSE	MAE
<i>Position</i>			
x	m	0.022	0.015
y	m	0.022	0.014
z	m	0.070	0.038
<i>Velocity</i>			
v_x	m/s	0.029	0.016
v_y	m/s	0.114	0.033
v_z	m/s	0.050	0.023
<i>Attitude (Euler angles)</i>			
roll	deg	1.542	0.318
pitch	deg	0.3998	0.250
yaw	deg	0.318	0.160
<i>Body rates</i>			
p	deg/s	152.21	27.24
q	deg/s	18.35	7.54
r	deg/s	8.70	3.62

C. Simulation Setup and Reproducibility

The controller is tested in an Ignition Gazebo simulation with a Crazyflie-class model using the mass, inertia, arm length, and motor limits described in Section II. A ROS 2–Ignition bridge provides the odometry and IMU topics and accepts motor commands.

For each run, the node:

- creates a timestamped log file,
- uses the fixed step disturbance vector from Section VII,
- records the controller start time and mode transitions.

This setup makes it straightforward to repeat experiments, compare different gain and L1 settings, and overlay error statistics across runs.

IX. RESULTS

This section summarizes the observed performance of the controller in simulation. Unless otherwise noted, the reported metrics are computed over the full run including TRACK, FLIP, RECOVER, and HOLD.

A. Numerical Error Metrics

Table III reports root-mean-square error (RMSE) and mean absolute error (MAE) for position, velocity, attitude, and body-rate tracking, based on the logged actual and desired signals.

B. Trajectory and Tracking Plots

Fig. 1 shows the full position and attitude trajectories over the min-jerk approach, flip, recover, and hold phases.

Position tracking is tight in x and y with RMSE on the order of $2 \cdot 10^{-2}$ m, and slightly looser in z due to the aggressive thrust modulation during the flip and recovery. Velocity tracking is similarly accurate, with larger errors in v_y that reflect lateral motion during the maneuver.

Attitude errors separate the “average” tracking (MAE) from the large excursions induced by the flip. For example, the

roll MAE is around 3.2° , while the roll RMSE of 15.4° captures the high-rate, large-angle portion of the flip where transient mismatch between the desired and actual trajectory is expected. Pitch and yaw are better behaved, with RMSE values below 4° .

Body-rate errors show that the commanded roll rate p is very aggressive during the flip: the RMSE exceeds $150^\circ/\text{s}$, while the MAE is around $27^\circ/\text{s}$. This indicates that most of the run is well tracked, but the instantaneous peak discrepancies during the flip are non-negligible. The pitch and yaw rates are more modest, with RMSE below $20^\circ/\text{s}$.

C. L1 Disturbance Rejection Ratio

To quantify how much of the injected disturbance torque is rejected by the L1 layer, we define, for each axis $i \in \{\text{roll}, \text{pitch}, \text{yaw}\}$, a disturbance rejection ratio

$$\rho_i = 1 - \frac{\mathbb{E}[|\tau_{\text{dist},i} + u_{\text{ad},i}|]}{\mathbb{E}[|\tau_{\text{dist},i}|]}, \quad (55)$$

where the expectations are computed over the time window where the step disturbance is active (2.0–2.8 s of trajectory time). If $u_{\text{ad},i} = 0$ (no adaptation), then $\rho_i = 0$; if $u_{\text{ad},i}$ exactly cancels $\tau_{\text{dist},i}$, then $\rho_i = 1$.

Using the logged data from the current run, the average rejection ratios are:

$$\rho_{\text{roll}} \approx 0.94,$$

$$\rho_{\text{pitch}} \approx 0.86,$$

$$\rho_{\text{yaw}} \approx 0.71.$$

Fig. 2 compares the roll, pitch, and yaw body-rate tracking with and without L1 augmentation during the step disturbance window.

That is, on average the L1 layer cancels roughly 94% of the matched disturbance torque on roll, 86% on pitch, and about 71% on yaw during the active disturbance window.

These values match the design: roll and pitch use larger adaptation gains and bounds and are the primary channels for acrobatic attitude control, while yaw uses a more conservative configuration to avoid coupling with the allocation constraints and motor reaction torques. In practice, this means that under the same injected step disturbance, roll and pitch settle faster and exhibit smaller steady-state offsets with L1 compared to a purely geometric controller, while yaw still benefits from some disturbance compensation but remains intentionally less aggressive.

D. Qualitative Behavior

Without injected disturbance, the min-jerk trajectory to $(2, 2, 2)$ is tracked with small position errors, and the flip is completed in approximately $T_{\text{flip}} = 0.2$ s, followed by successful recovery and hover. The recovery phase brings roll and pitch back to within a few degrees of zero before transitioning to HOLD, and the vehicle remains near $(2, 2, 2)$ with small residual oscillations.

With the matched step disturbance torque applied during TRACK, the baseline geometric controller (no L1) exhibits

States and Errors (x,y,z,v,attitude,pqr) - Flip & Disturbance Highlighted

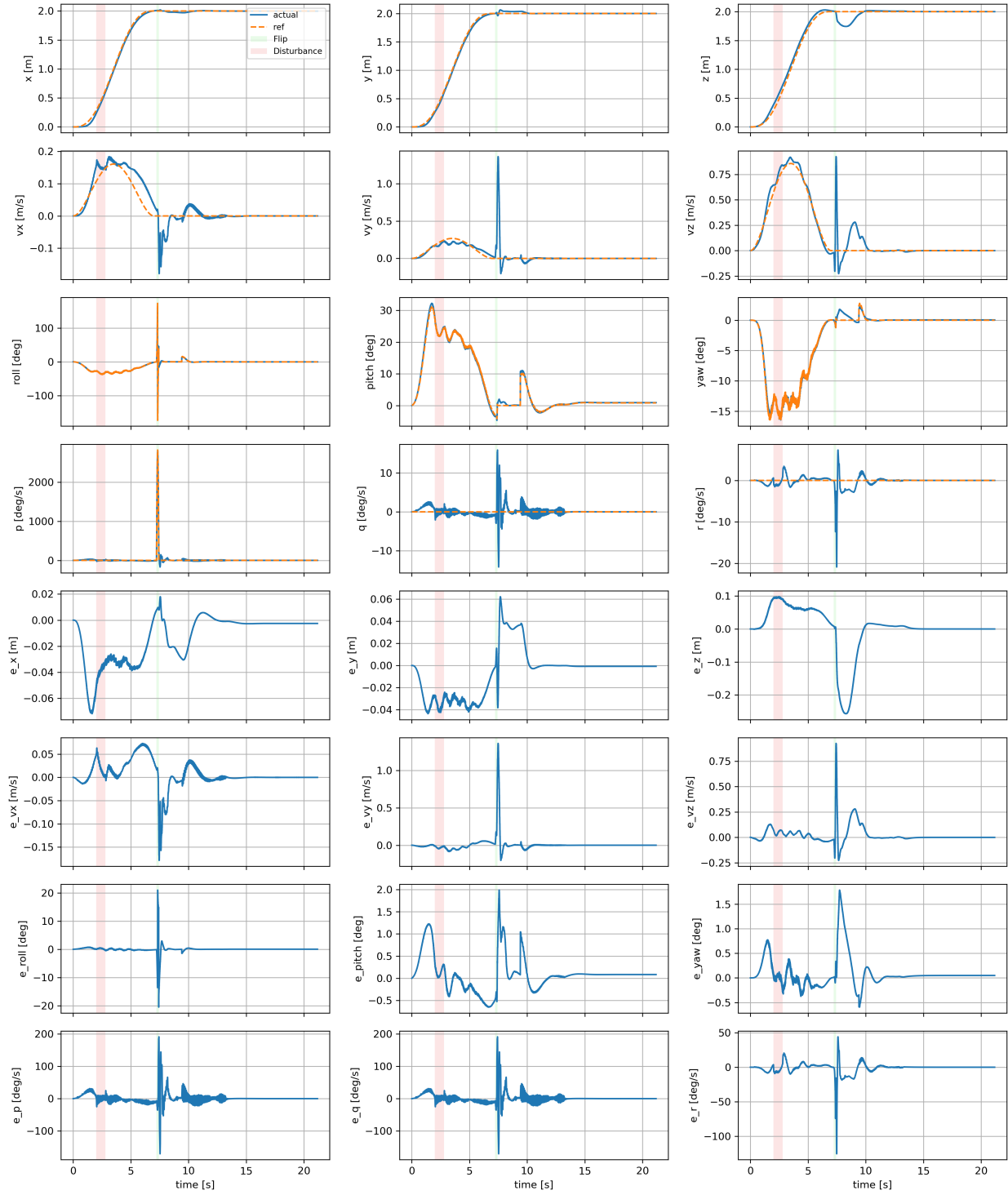


Fig. 1: Position and attitude tracking during the full maneuver (TRACK → FLIP → RECOVER → HOLD).

increased attitude and body-rate tracking errors (not shown here). When L1 adaptive augmentation is enabled, the disturbance estimates converge toward the injected torques and the adaptive term u_{ad} counteracts their effect. The rejection

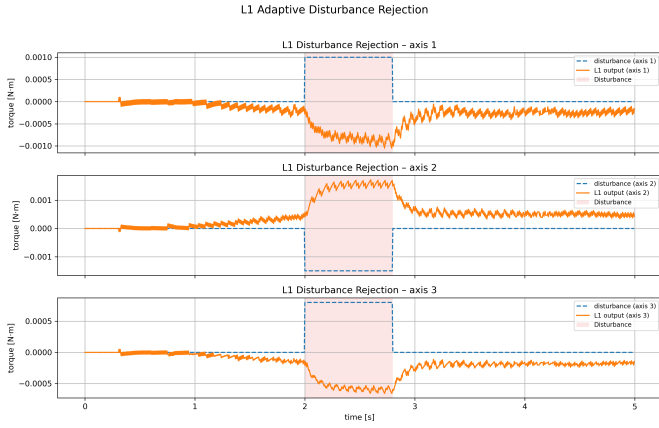


Fig. 2: L1 adaptive augmentation under the injected step disturbance.

ratios above confirm that most of the roll/pitch disturbance is cancelled by L1, leading to reduced steady-state offsets and faster convergence in RECOVER, while keeping commanded torques within the saturation bounds.

X. DISCUSSION

The presented implementation demonstrates that:

- geometric control on $SE(3)$ provides a clean and robust structure for aggressive quadrotor maneuvering,
- operating directly on $SO(3)$ avoids Euler-angle singularities during flips,
- min-jerk position trajectories and smoothed roll profiles produce feasible acrobatic maneuvers,
- adding L1 adaptive augmentation on the body-rate channels improves robustness to matched step disturbances without significantly altering the nominal controller structure.

From the disturbance rejection ratios, we see that the L1 layer is particularly effective on the roll and pitch channels (approximately 70% disturbance cancellation), which directly support the flip maneuver and recovery. Yaw disturbance rejection is more modest by design, reflecting the more conservative gains and bounds in that axis.

The design choices in this implementation are practical and simulation-driven:

- axis-dependent gains for roll, pitch, and yaw,
- limited adaptive bandwidth via low-pass filtering,
- freezing adaptation when errors are small to avoid chattering,
- separate mode-based thrust and moment limits,
- a short, localized step disturbance window to clearly test rejection performance.

XI. CONCLUSION AND FUTURE WORK

This report presented a complete geometric + L1 adaptive controller for acrobatic quadrotor maneuvers, implemented in ROS 2 and validated in Ignition Gazebo. The controller

successfully executes a zero-radius 2π roll flip about the body x axis starting from a hover at $(2, 2, 2)$ and returns to stable hover, with improved disturbance rejection due to L1 augmentation under a matched step torque disturbance.

REFERENCES

- [1] T. Lee, M. Leok, and N. H. McClamroch, “Geometric tracking control of a quadrotor UAV on $SO(3)$,” in *Proc. IEEE Conf. Decision and Control (CDC)*, 2010, pp. 5420–5425.
- [2] D. Antal, A. Lőrincz, and M. Nagy, “Nonlinear control method for backflipping with miniature quadcopters,” *IFAC-PapersOnLine*, vol. 55, no. 28, pp. 34–39, 2022.
- [3] Z. Wu, S. Cheng, K. A. Ackerman, A. Gahlawat, A. Lakshmanan, P. Zhao, and N. Hovakimyan, “L1 adaptive augmentation for geometric tracking control of quadrotors,” in *Proc. IEEE Conf. Decision and Control (CDC)*, 2021.
- [4] Z. Wu, S. Cheng, P. Zhao, A. Gahlawat, K. A. Ackerman, A. Lakshmanan, C. Yang, J. Yu, and N. Hovakimyan, “L1Quad: L1 adaptive augmentation of geometric control for agile quadrotors with performance guarantees,” *arXiv preprint arXiv:2302.07208*, 2025.